

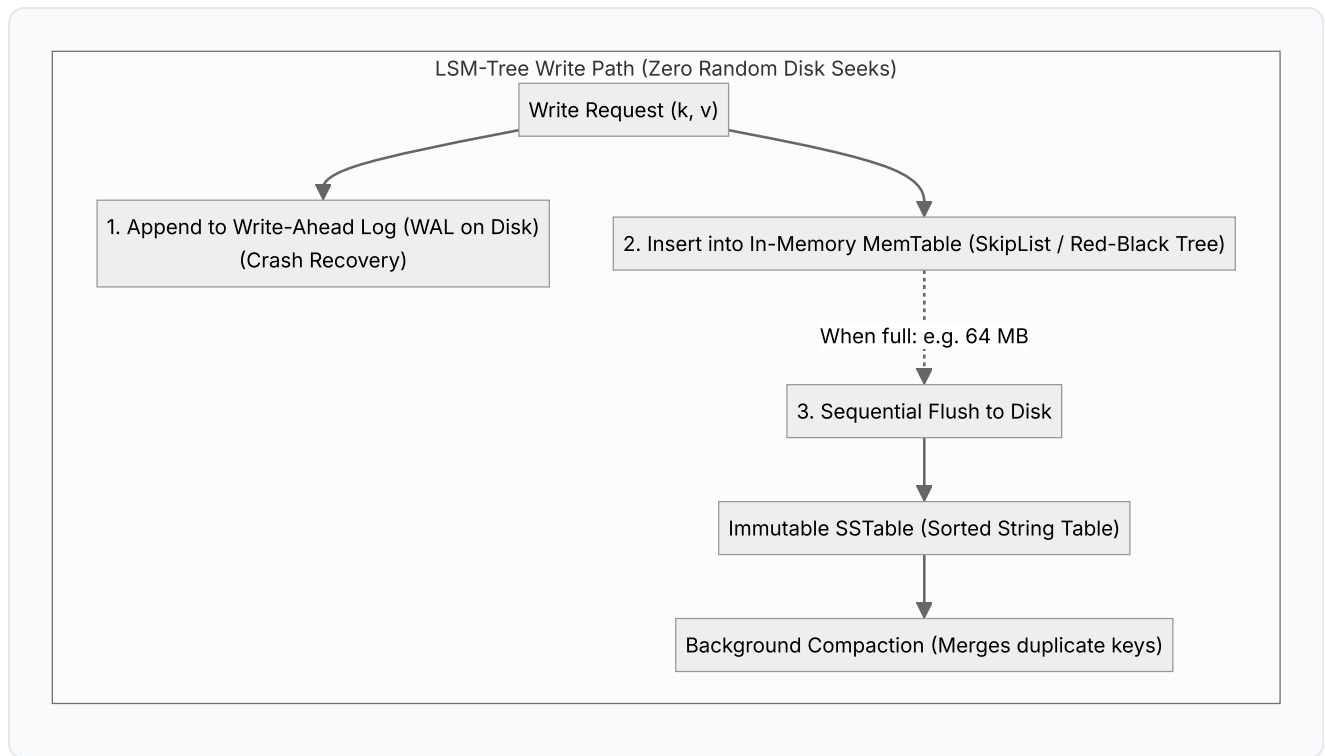
HLD Chapter 3: Databases, Storage Engines, Sharding & Consistent Hashing

Core Learning Objective: Master database architecture at planetary scale. Understand B-Trees vs LSM-Trees, replication topologies, horizontal partitioning (sharding), and the mathematics of Consistent Hashing with virtual nodes.

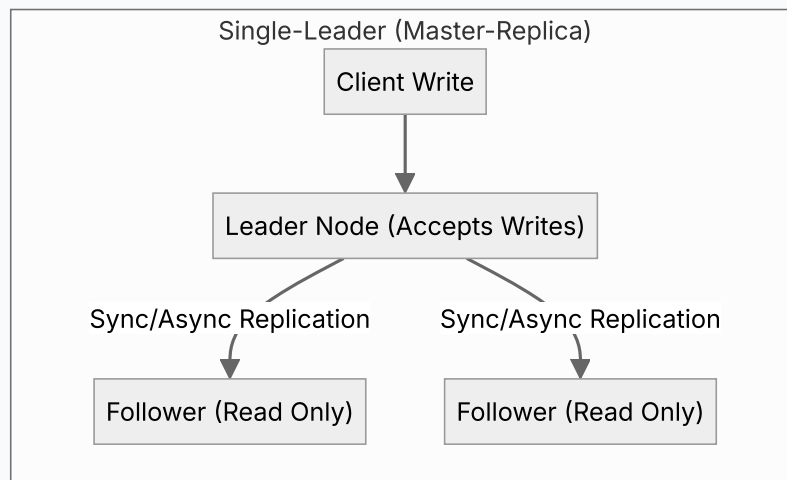
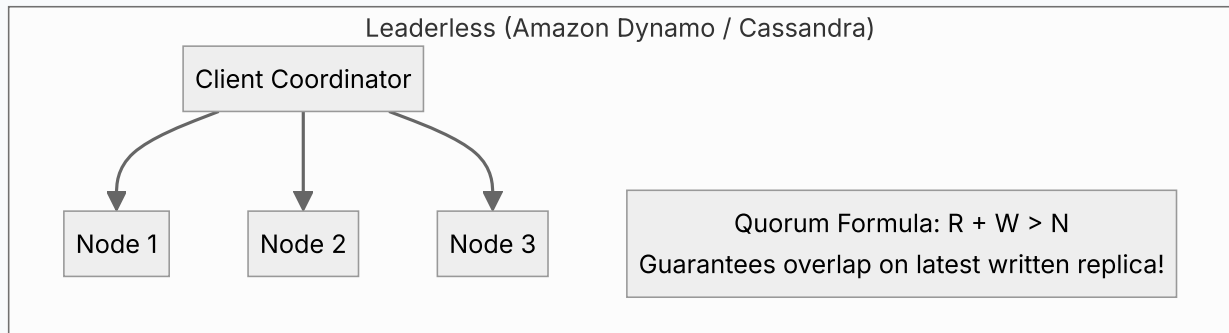
1. Storage Engines Under the Hood: B-Trees vs LSM-Trees

Every database's performance characteristics are dictated by its underlying on-disk storage engine:

Metric	B-Tree (PostgreSQL, MySQL InnoDB)	LSM-Tree (Cassandra, RocksDB, ScyllaDB)
Primary Optimization	Read-Heavy workloads	Write-Heavy workloads
Write Mechanism	In-place random page updates	Append-only sequential writes to MemTable + WAL
Storage Structure	Self-balancing tree of fixed-size disk blocks (4KB – 16KB)	Memory buffer (MemTable) flushed to sorted disk files (SSTables)
Write Amplification	Higher (full page rewrite on single row update)	Lower for writes; compaction overhead later
Lookup Latency	$O(\log N)$ predictable disk seeks	In-memory MemTable check → Bloom Filter check → SSTable seek



2. Replication Models: Leader vs Leaderless



The Quorum Mathematics ($R + W > N$)

In leaderless systems:

- N = Number of replicas stored for each key.
- W = Number of write acknowledgements required before reporting success.
- R = Number of replicas queried on read.
- If $R + W > N$: By the Pigeonhole Principle, at least **one** of the queried nodes in R is guaranteed to contain the most recent write.

3. Sharding & Horizontal Partitioning

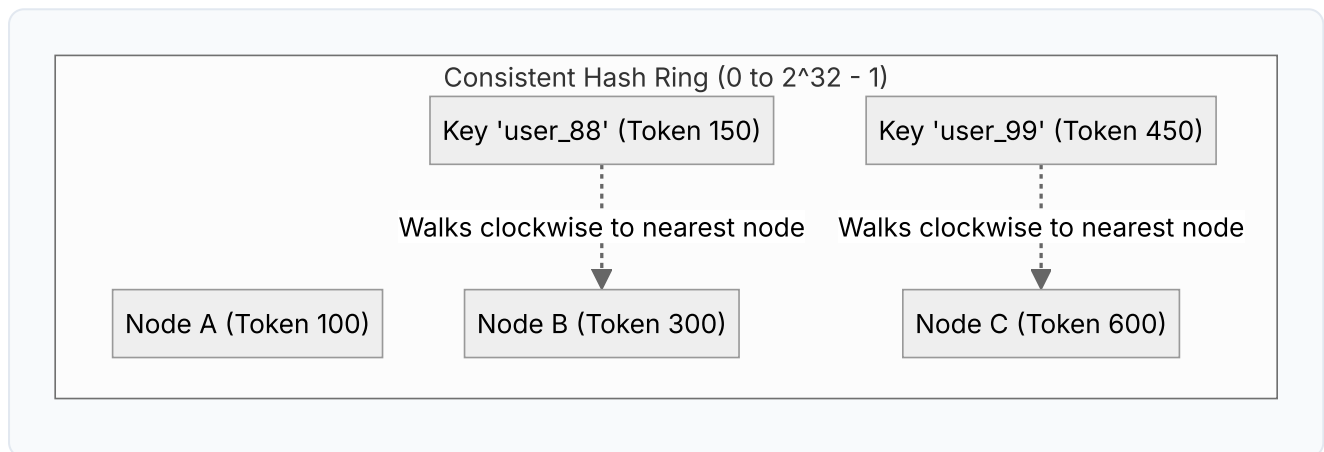
When data exceeds the disk capacity or throughput limits of a single machine (> 2 TB or $> 20,000$ QPS), the database must be partitioned:

Partitioning Strategies:

1. **Range-Based Partitioning:** (e.g. Users $A - C$ on Node 1, $D - F$ on Node 2).
 - *Flaw:* Causes massive **hot spots** (e.g. Celebrity accounts or localized surges).
2. **Hash-Based Partitioning:** $\text{Node} = \text{hash}(\text{key}) \pmod{N}$.
 - *Fatal Flaw:* When a node is added or removed ($N \rightarrow N + 1$), nearly 100% of keys hash to a new node, causing a catastrophic full-cluster re-indexing storm!

4. Consistent Hashing with Virtual Nodes (The Standard)

Consistent Hashing maps both **nodes** and **data keys** to an abstract circular ring ($[0, 2^{32} - 1]$).



Why It Eliminates Re-indexing Storms

When a new node is inserted into the ring, **only the keys immediately preceding the new node are moved**; all other nodes remain completely untouched! On average, only $\frac{K}{N}$ keys are relocated.

Eliminating Hotspots: Virtual Nodes (vnodes)

If physical nodes are placed randomly on the ring, load distribution is unequal. **Solution:** Each physical server is assigned multiple **Virtual Nodes** (e.g., 256 tokens scattered across the ring: `Node_A#1`, `Node_A#2`, etc.). This ensures statistical uniformity across all physical hardware.

Complete Consistent Hash Ring Implementation in Python



5. Runnable Models: Quorums and Consistent Hashing

Quorum arithmetic: when is a read guaranteed to see the latest write?

With N replicas, a write acknowledged by W and a read that asks R replicas, $R + W > N$ guarantees the read set overlaps the write set in at least one replica:

Tuning: $W = N, R = 1$ gives fast reads and fragile writes; $W = 1, R = N$ does the opposite; $W = R = \text{majority}$ balances them and tolerates a minority of failures. Quorum overlap alone does not give linearizability (concurrent writes, clock issues and sloppy quorums can still surprise you); it gives a bound on staleness.

Consistent hashing: adding a node moves only about $1/n$ of the keys

Virtual nodes (here 100 per physical node) do two jobs: they even out the load, and they let a failed node's keys spread across many survivors instead of dumping onto one neighbour.

Choosing a replication and partitioning scheme

Need	Pick	Watch out for
Strong consistency, modest scale	Single-leader replication, synchronous follower	Leader is a write bottleneck; failover must avoid split brain
High write availability across regions	Multi-leader or leaderless with quorums	Conflict resolution (last-write-wins loses data; CRDTs or app-level merge preserve it)
Even load, simple lookups	Hash partitioning	Range queries must scatter-gather
Range scans by key	Range partitioning	Hot ranges (monotonic keys); split and rebalance
Elastic cluster	Consistent hashing with virtual nodes	Rebalancing traffic while data moves

Further Reading

- [PostgreSQL: high availability and replication](#)

- [MongoDB sharding](#)
 - [DynamoDB partition key design](#)
-

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. Replication versus partitioning (sharding)?

Replication copies the same data for availability and read scale; partitioning splits different data across nodes for write and storage scale.

▼ 2. What makes a good shard key?

High cardinality and even distribution of load, aligned with the dominant query pattern; avoid monotonic keys that create hot shards.

▼ 3. What is read-your-writes consistency?

A client always sees its own writes, e.g. by reading from the primary or sticky routing after a write.

▼ 4. Why does a cross-shard transaction hurt?

It needs coordination (2PC or sagas), adding latency and failure modes.